

Checkpoint: SHAP Integration

Real-Time User Behavior Anomaly Detection Platform — Amen Bank

1. Objective

The objective of this phase was to provide human-readable explanations for each alert generated by the Decision Service. Rather than presenting raw anomaly scores, the system now answers the supervisor's core question: *why was this alert raised?*

2. Explanation Architecture

The explanation system follows a **trigger-aware** design. Since alerts can originate from three distinct mechanisms — ML scoring, regulatory rules, or both — the explanation adapts its content accordingly:

- **ML-triggered alerts:** Explanations derived from model internals (SHAP for the Autoencoder, expected-vs-actual for the LSTM).
- **Rule-triggered alerts:** The regulatory rule description itself serves as the explanation (e.g., "Amount exceeds BCT 10,000 DT reporting threshold").
- **Combined triggers:** Both ML and rule explanations are assembled together.

3. Model-Specific Explanation Strategies

A key design decision was to use **different explanation strategies** for each model, reflecting their fundamentally different architectures and the questions they answer.

3.1 Autoencoder — SHAP (KernelExplainer)

The Autoencoder takes a single 30-dimensional enriched feature vector and reconstructs it. The anomaly score is the mean squared reconstruction error. SHAP attributes this score across the 30 input features, answering: *which attributes of this specific event are unusual?*

Why KernelExplainer over DeepExplainer

The anomaly score is computed as $(\text{reconstruction} - \text{input})^2$, which involves subtracting the input from the output. This dependency on the input itself prevents DeepExplainer from tracing the computation graph as a standard forward pass. KernelExplainer, being model-agnostic (numpy-in, numpy-out), handles this naturally.

Background Dataset

SHAP requires a reference baseline of normal events to determine feature contributions. A sample of 100 scaled normal events is drawn from the enriched dataset and saved as `models/shap_background.npy`. This background is regenerated alongside the weekly model retrain to maintain consistency with the current scaler.

Composition Pattern

The AEScorer wrapper class holds a reference to the existing trained Autoencoder instance rather than loading a separate copy. This avoids model duplication in memory, as ScoreFusion already loads both models. The scorers receive model instances via constructor injection.

Output format:

```
{
  "score": 78.62,
  "top_features": {
    "amount_to_mean_ratio": (shap=-10.17, value=13.60),
    "is_near_cheque_ceiling": (shap=-4.70, value=1.0)
  }
}
```

3.2 LSTM — Expected vs. Actual

The LSTM predicts the next event's 30 features from a sequence of 10 previous events. Rather than applying SHAP — which would produce hundreds of attributions across `seq_len x n_features` — the explanation is derived directly from the model's output: *what did the model expect vs. what actually happened?*

This approach answers: *how does this event break the client's recent behavioral sequence?*

Feature Whitelist

Not all 30 features produce meaningful expected-vs-actual explanations. Binary features (e.g., `is_home_branch`, `op_retrait`) yield unintuitive predictions like "expected 0.37, observed 1.0." Similarly, `z_amount` is excluded because non-technical supervisors do not understand z-scores, and `account_age_days` does not vary between events. The whitelist retains 11 continuous, interpretable features:

```
LSTM_EXPLAINABLE_FEATURES = [
    "amount", "hour", "amount_to_mean_ratio",
    "op_type_probability", "day_distance_from_preferred",
    "tx_count_24h", "tx_count_7d", "cumulative_amount_24h",
    "near_threshold_count_7d", "employee_tx_count_24h",
    "same_employee_client_count_24h"
]
```

Ranking in Scaled Space, Display in Raw Space

Per-feature deviations are ranked using squared error in scaled space (ensuring fair comparison across features), but the reported expected/actual values are inverse-transformed to raw space for supervisor readability.

4. Template-Based Natural Language Generation

Rather than using an LLM or learned text generation — which would introduce complexity, latency, and security concerns in a banking environment — explanations are rendered through a **deterministic template dictionary**. Each feature maps to a pre-written sentence with placeholders:

```
AE_TEMPLATES = {
    "amount_to_mean_ratio":
        "Amount is {value}x the client's usual average",
    ...
}
```

```
LSTM_TEMPLATES = {
    "amount":
        "Expected amount ~{expected} DT, observed {actual} DT",
    ...
}
```

This approach is auditable, deterministic, and requires no external API dependencies.

5. Integration with Decision Service

The explanation assembly is implemented as **Layer 4** of the Decision Service, executed after tier assignment, rule escalation, and risk history adjustment. The trigger type is determined by inspecting whether the fused score exceeded the T1 threshold and/or regulatory rules fired.

Explanations are only assembled for alerts above INFO tier. The final output includes both a reasons list (human-readable sentences for the supervisor) and a raw field (original numerical data for audit compliance).

Data Flow

- **Scoring Service** runs inference, computes AE SHAP values and LSTM expected-vs-actual, selects top-k contributors, and attaches the raw explanation data to the scored event.
- **Decision Service** receives the scored event with explanation data, determines the alert tier, identifies the trigger type, applies templates, and packages the assembled explanation into the final Decision output.

6. End-to-End Validation

The system was validated on a cheque structuring anomaly (Law n°41-2024). A client who normally transacts ~700 DT submitted a cheque for 27,561 DT near the 30,000 DT legal ceiling. The supervisor explanation produced:

- **AE:** "Amount is 13.6x the client's usual average"
- **LSTM:** "Expected amount ~697 DT, observed 27,561 DT" — "Expected 3.58 days from payday, observed 12.0 days"
- **Rules:** "Amount 27,561 DT >= 10,000 DT reporting threshold (BCT)" — "Cheque amount 27,561 DT near 30,000 DT ceiling (Law n°41-2024)"

Three independent mechanisms converged on the same event, each providing a complementary perspective: per-feature anomaly (AE), behavioral sequence break (LSTM), and regulatory compliance (rules).

7. New Artifacts

File	Description
src/scoring/ae_scorer.py	AE wrapper with SHAP KernelExplainer
src/scoring/lstm_scorer.py	LSTM wrapper with expected-vs-actual

src/decision/explainer.py	Template dictionaries + assemble_explanation
src/decision/decision.py	Updated with Layer 4 + explanation fields
tests/test_shap_e2e.py	End-to-end validation script
models/shap_background.npy	SHAP background dataset (100 scaled normal events)

8. Next Phase

The next phase is the **streaming pipeline** using RedPanda, which will connect the services (Ingestion → Enrichment → Scoring → Decision) through the four-topic architecture: raw-events → enriched-events → scored-events → decisions.